

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO3: Analyze knowledge representation and reasoning using propositional logic, FOL, inference, resolution, chaining methods, knowledge representation to perform logical reasoning for drawing valid conclusions and decision making. (BL4)

Assessment Tool Weightage: 50%

Time Duration: 1 Hour

QUESTIONS: 4

Total Marks:40

Annexure A

CASE STUDY

Code of Conduct:

*I **NEELAM NANDINI [192472277]** (Name / Reg No) certify that this certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: N. Nandini.

Annexure A

CASE STUDY

Case Study 1 — Smart Medical Diagnosis System

A hospital is designing a rule-based Logical Agent for preliminary patient diagnosis. The system's knowledge base contains:

- I. If a patient has Fever AND Cough $\rightarrow$ Possible Flu**
- II. If a patient has Fever AND Rash $\rightarrow$ Possible Measles**
- III. If a patient has Cough AND Breathlessness $\rightarrow$ Possible Pneumonia**
- IV. Facts about Patient A: Fever = True, Cough = True, Breathlessness = True**

(a) Represent the above rules and facts precisely using Propositional Logic symbols. Define each symbol used.

Let the following atomic propositions represent the clinical symptoms and diagnoses used by the hospital's rule-based logical agent: let F denote "the patient has Fever", C denote "the patient has Cough", R denote "the patient has Rash", B denote "the patient has Breathlessness", Flu denote "the patient possibly has Flu", $Measles$ denote "the patient possibly has Measles", and $Pneumonia$ denote "the patient possibly has Pneumonia". Each of these symbols is a Boolean proposition that can be assigned the truth value True or False depending on whether the corresponding symptom or diagnosis holds for a given patient.

Using these symbols and the standard propositional connectives $\wedge$ (conjunction, AND) and $\rightarrow$ (material implication, IF...THEN), the three rules in the hospital's knowledge base can be written precisely as: Rule 1: $F \wedge C \rightarrow Flu$; Rule 2: $F \wedge R \rightarrow Measles$; Rule 3: $C \wedge B \rightarrow Pneumonia$. Each rule states that whenever the conjunction on the left-hand side (the antecedent) is true, the proposition on the right-hand side (the consequent) may be inferred as true. The implication connective does not assert any temporal or causal relationship; it purely captures a logical dependency that the diagnostic agent can use for automated reasoning.

The facts supplied about Patient A are also propositions with assigned truth values: $F = \text{True}$, $C = \text{True}$, and $B = \text{True}$. The knowledge base does not explicitly state a value for R (Rash), and under the closed-world assumption commonly used in rule-based expert systems, any proposition not asserted as true is treated as false or unknown for inference purposes. This precise symbolic representation is essential because it converts an informal, natural-language description of medical knowledge into a formal structure that a logical inference engine can process automatically, ensuring that the diagnostic conclusions the system reaches are logically sound and can be traced back, step by step, to the original rules and facts.

(b) Apply Forward Chaining on the knowledge base to derive all possible diagnoses for Patient A. Show each inference step with the rule fired and fact derived.

Forward chaining is a data-driven inference technique in which the reasoning engine starts from the known facts and repeatedly fires any rule whose antecedent is fully satisfied, adding the newly derived conclusion to the set of known facts, until no further rule can fire. For Patient A, the initial fact base is $\{F = \text{True}, C = \text{True}, B = \text{True}\}$, with R remaining unasserted.

Step 1: The engine examines Rule 1 ($F \wedge C \rightarrow \text{Flu}$). Both F and C are true in the fact base, so the antecedent is satisfied. The rule fires and the new fact "Flu = True" (Possible Flu) is added to the knowledge base.

Step 2: The engine examines Rule 2 ($F \wedge R \rightarrow \text{Measles}$). F is true, but R is not present in the fact base, so the antecedent is not satisfied. Rule 2 cannot fire and Measles is not derived.

Step 3: The engine examines Rule 3 ($C \wedge B \rightarrow \text{Pneumonia}$). Both C and B are true, so the antecedent is satisfied. The rule fires and the new fact "Pneumonia = True" (Possible Pneumonia) is added to the knowledge base.

Step 4: The engine performs another pass over all rules to check whether any newly derived fact enables further firing. Since Flu and Pneumonia are terminal conclusions that do not appear as antecedents of any other rule, no additional rule can fire, and the system has reached a fixed point.

Final derived diagnoses for Patient A: Possible Flu and Possible Pneumonia. Possible Measles is not supported because the Rash symptom was never confirmed. This demonstrates how forward chaining exhaustively and systematically expands the fact base until all logically justified conclusions have been reached.

(c) The goal is to confirm 'Patient A has Pneumonia.' Apply Backward Chaining and draw the goal reduction tree to verify this goal.

Backward chaining is a goal-driven inference technique that starts from the goal to be proved and works backwards, decomposing the goal into subgoals using the rules whose consequent matches the goal, until each subgoal is either verified against a known fact or shown to be unprovable. Here the goal is to confirm the proposition "Patient A has Pneumonia".

The reasoning engine searches the knowledge base for a rule whose consequent is Pneumonia and finds Rule 3: $C \wedge B \rightarrow \text{Pneumonia}$. To prove Pneumonia, the engine must therefore prove both subgoals C (Cough) and B (Breathlessness), since they are joined by conjunction and both must hold.

Subgoal C is checked against the fact base: $C = \text{True}$ is directly asserted as a fact about Patient A, so this subgoal is immediately satisfied and becomes a leaf node of the reduction tree. Subgoal B is checked next: $B = \text{True}$ is also directly asserted as a fact, so this subgoal is likewise satisfied and becomes a leaf node.

Goal reduction tree: Pneumonia (root) is reduced by Rule 3 into two AND-connected child subgoals, C and B; both children are leaf facts confirmed True in the knowledge base. Since every leaf under the AND-node evaluates to True, the conjunction $C \wedge B$ evaluates to True, and by Rule 3 the parent goal Pneumonia is therefore proved True.

Because both subgoals are satisfied without any need for further decomposition, the backward-chaining procedure terminates successfully and confirms the goal: Patient A indeed has Possible Pneumonia, consistent with the forward-chaining result obtained in part (b). This illustrates the complementary nature of forward and backward chaining: the former exhaustively derives everything that can be known, while the latter efficiently verifies a single specific query by working only through the rules relevant to that goal.

Case Study 2 — Autonomous Traffic Management Agent

A smart city traffic controller uses a FOL-based knowledge base to manage emergency vehicle routing:

- I. $x: \text{Vehicle}(x) \wedge \text{EmergencyType}(x) \rightarrow \text{ClearPath}(x)$ [Rule 1]
- II. $\forall \forall \forall x: \forall \text{ClearPath}(x) \wedge \rightarrow \wedge \text{GreenSignal}(x) \wedge \wedge \neg \text{RedSignal}(x)$ [Rule 2]
- III. $x \ y: \text{Vehicle}(x) \wedge \text{Behind}(x,y) \wedge \text{GreenSignal}(y) \rightarrow \text{Proceed}(x)$ [Rule 3]
- IV. **Facts:** $\text{Vehicle}(\text{Ambulance}), \text{EmergencyType}(\text{Ambulance}), \text{Vehicle}(\text{CarA}), \text{Behind}(\text{CarA}, \text{Ambulance})$

(a) Apply Unification to match Rule 1 with the given facts. Show the complete substitution set θ (MGU) at each step.

Unification is the process of finding a substitution θ that makes two first-order logic expressions syntactically identical. Rule 1 in the traffic knowledge base is $\forall x: \text{Vehicle}(x) \wedge \text{EmergencyType}(x) \rightarrow \text{ClearPath}(x)$. To apply this rule, the unification algorithm must match its antecedent literals, $\text{Vehicle}(x)$ and $\text{EmergencyType}(x)$, against the asserted facts $\text{Vehicle}(\text{Ambulance})$ and $\text{EmergencyType}(\text{Ambulance})$.

Step 1: Unify $\text{Vehicle}(x)$ with $\text{Vehicle}(\text{Ambulance})$. The predicate symbols match ($\text{Vehicle} = \text{Vehicle}$) and there is one argument pair: x (a variable) against Ambulance (a constant). Since x is a variable, it is bound to the constant, giving the substitution $\theta_1 = \{x/\text{Ambulance}\}$.

Step 2: Apply θ_1 to the second antecedent literal $\text{EmergencyType}(x)$, producing $\text{EmergencyType}(\text{Ambulance})$, and unify this with the fact $\text{EmergencyType}(\text{Ambulance})$. The predicate symbols and the single constant argument match exactly, so no new binding is required and the substitution remains consistent: $\theta = \{x/\text{Ambulance}\}$.

Because both antecedent literals unify consistently under the same substitution, $\theta = \{x/\text{Ambulance}\}$ is the most general unifier (MGU) for Rule 1 against the given facts; it is the least restrictive substitution that achieves unification, since it binds only the single variable present and does not over-specify any other term.

Step 3: Apply the MGU to the consequent of Rule 1, $\text{ClearPath}(x)$, yielding $\text{ClearPath}(\text{Ambulance})$. This grounded literal is then asserted as a new fact in the knowledge base, meaning the system has logically established that a clear path must be provided for the ambulance. This unification process is the mechanism that allows a general, variable-based rule to be applied to specific, concrete objects during automated reasoning.

(b) Using the Resolution method, prove that ‘Proceed (CarA)’ is true. Convert relevant clauses to CNF and show each resolution step.

To prove $\text{Proceed}(\text{CarA})$ by resolution, all relevant sentences must first be converted into Conjunctive Normal Form (CNF), since resolution operates purely on disjunctive clauses. Rule 1 becomes the clause $\neg\text{Vehicle}(x) \vee \neg\text{EmergencyType}(x) \vee \text{ClearPath}(x)$. Rule 2, which has a conjunctive consequent, is split into two separate clauses: $\neg\text{ClearPath}(x) \vee \text{GreenSignal}(x)$ and $\neg\text{ClearPath}(x) \vee \neg\text{RedSignal}(x)$. Rule 3 becomes $\neg\text{Vehicle}(x) \vee \neg\text{Behind}(x,y) \vee \neg\text{GreenSignal}(y) \vee \text{Proceed}(x)$. The four facts, $\text{Vehicle}(\text{Ambulance})$, $\text{EmergencyType}(\text{Ambulance})$, $\text{Vehicle}(\text{CarA})$ and $\text{Behind}(\text{CarA}, \text{Ambulance})$, are already unit clauses. Finally, the goal is negated to enable proof by refutation: $\neg\text{Proceed}(\text{CarA})$.

Resolution step 1: Resolve Rule 1 (with $x/\text{Ambulance}$) against the fact $\text{Vehicle}(\text{Ambulance})$, cancelling the complementary literals $\neg\text{Vehicle}(\text{Ambulance})$ and $\text{Vehicle}(\text{Ambulance})$, leaving $\neg\text{EmergencyType}(\text{Ambulance}) \vee \text{ClearPath}(\text{Ambulance})$.

Resolution step 2: Resolve the result of step 1 against the fact $\text{EmergencyType}(\text{Ambulance})$, cancelling the complementary literals, leaving the unit clause $\text{ClearPath}(\text{Ambulance})$.

Resolution step 3: Resolve $\text{ClearPath}(\text{Ambulance})$ against the first split of Rule 2, $\neg\text{ClearPath}(x) \vee \text{GreenSignal}(x)$ (with $x/\text{Ambulance}$), cancelling the complementary literals and leaving the unit clause $\text{GreenSignal}(\text{Ambulance})$.

Resolution step 4: Take Rule 3 with the substitution x/CarA , $y/\text{Ambulance}$, giving $\neg\text{Vehicle}(\text{CarA}) \vee \neg\text{Behind}(\text{CarA}, \text{Ambulance}) \vee \neg\text{GreenSignal}(\text{Ambulance}) \vee \text{Proceed}(\text{CarA})$. Resolve this in turn against the facts $\text{Vehicle}(\text{CarA})$, $\text{Behind}(\text{CarA}, \text{Ambulance})$, and against $\text{GreenSignal}(\text{Ambulance})$ derived in step 3, cancelling one complementary literal at each stage, and finally leaving the unit clause $\text{Proceed}(\text{CarA})$.

Resolution step 5: Resolve the derived clause $\text{Proceed}(\text{CarA})$ against the negated goal $\neg\text{Proceed}(\text{CarA})$. The two literals are exact complements, so they cancel completely, producing the empty clause $\square$. The derivation of the empty clause is, by the completeness of the resolution procedure, a proof that the original set of clauses (KB plus negated goal) is unsatisfiable, which in turn proves that $\text{Proceed}(\text{CarA})$ logically follows from the knowledge base.

(c) Analyze whether the current FOL knowledge base is sufficient to handle two simultaneous emergency vehicles. Identify what additional rules or facts are needed and justify logically.

The current FOL knowledge base is not sufficient to correctly handle two simultaneous emergency vehicles. Each of the three rules is defined purely in terms of a single vehicle variable x (or the pair x, y for the Behind relation) and contains no predicate that compares, ranks, or arbitrates between two different emergency vehicles that might require conflicting resources at the same time, such as the same intersection or the same green signal.

If two ambulances, for example Ambulance1 and Ambulance2 , are both instantiated as EmergencyType and both approach the same intersection from different directions, Rule 1 and Rule 2 would independently derive ClearPath and GreenSignal for both vehicles simultaneously. Because the knowledge base has no notion of mutual exclusion, it could logically sanction two conflicting green signals at a single intersection at the same time, which is unsafe and physically impossible to execute in the real traffic system.

To resolve this gap, the knowledge base needs additional predicates and facts such as $\text{Priority}(x, n)$ to encode a numeric or categorical priority level for each emergency vehicle (for instance, based on vehicle type, distance to the intersection, or dispatch time), $\text{IntersectionConflict}(x, y)$ to represent that two vehicles are approaching the same junction, and $\text{ApproachTime}(x, t)$ to represent estimated arrival time. A new rule would then be required, for example: $\text{Vehicle}(x) \wedge \text{Vehicle}(y) \wedge$

$\text{IntersectionConflict}(x,y) \wedge \text{Priority}(x,p_x) \wedge \text{Priority}(y,p_y) \wedge (p_x > p_y) \rightarrow \text{GreenSignal}(x) \wedge \text{HoldSignal}(y)$, which explicitly arbitrates between the two competing requests.

In summary, while the existing rules correctly handle the case of a single emergency vehicle, they lack the comparative and temporal predicates necessary for conflict resolution among multiple simultaneous emergencies; extending the KB with priority, conflict-detection, and arbitration rules is logically necessary before the system can be considered complete and safe for real-world deployment.

Case Study 3 — Agricultural Expert Reasoning System

An agricultural advisory logical agent operates with the following propositional knowledge base (KB):

- 1) P1: $\text{SoilDry} \rightarrow \text{IrrigationNeeded}$**
- 2) P2: $\text{IrrigationNeeded} \wedge \text{CropWheat} \rightarrow \text{ApplyDripMethod}$**
- 3) P3: $\neg \text{ApplyDripMethod} \wedge \text{CropWheat} \rightarrow \text{CropAtRisk}$**
- 4) Facts: $\text{SoilDry} = \text{True}, \text{CropWheat} = \text{True}$**

(a) Convert all sentences P1, P2, P3 and the facts into Conjunctive Normal Form (CNF). Show each conversion step (eliminate implication $\rightarrow$ move $\neg$ inward $\rightarrow$ distribute).

Converting a knowledge base to Conjunctive Normal Form (CNF) is a mechanical three-stage process: first, every implication is eliminated by rewriting $P \rightarrow Q$ as $\neg P \vee Q$; second, negation is moved inward using De Morgan's laws so that it applies only to atomic propositions; and third, disjunctions are distributed over conjunctions so that the final expression is a conjunction of clauses (disjunctions of literals).

P1: $\text{SoilDry} \rightarrow \text{IrrigationNeeded}$. Eliminating the implication gives $\neg \text{SoilDry} \vee \text{IrrigationNeeded}$. This expression already contains a single disjunction of literals with no further negation to move or conjunction to distribute, so it is already in CNF as one clause: $(\neg \text{SoilDry} \vee \text{IrrigationNeeded})$.

P2: $\text{IrrigationNeeded} \wedge \text{CropWheat} \rightarrow \text{ApplyDripMethod}$. Eliminating the implication gives $\neg(\text{IrrigationNeeded} \wedge \text{CropWheat}) \vee \text{ApplyDripMethod}$. Applying De Morgan's law to move the negation inward across the conjunction gives $(\neg \text{IrrigationNeeded} \vee \neg \text{CropWheat}) \vee \text{ApplyDripMethod}$, which simplifies directly to the single CNF clause $(\neg \text{IrrigationNeeded} \vee \neg \text{CropWheat} \vee \text{ApplyDripMethod})$, since disjunction is associative.

P3: $\neg \text{ApplyDripMethod} \wedge \text{CropWheat} \rightarrow \text{CropAtRisk}$. Eliminating the implication gives $\neg(\neg \text{ApplyDripMethod} \wedge \text{CropWheat}) \vee \text{CropAtRisk}$. Applying De Morgan's law gives $(\neg\neg \text{ApplyDripMethod} \vee \neg \text{CropWheat}) \vee \text{CropAtRisk}$, and double-negation elimination simplifies $\neg\neg \text{ApplyDripMethod}$ to ApplyDripMethod , yielding the CNF clause $(\text{ApplyDripMethod} \vee \neg \text{CropWheat} \vee \text{CropAtRisk})$.

Finally, the two given facts are unit clauses that require no conversion, since a single positive literal is trivially in CNF: (SoilDry) and (CropWheat) . The complete CNF knowledge base is therefore the conjunction of five clauses: $(\neg \text{SoilDry} \vee \text{IrrigationNeeded})$, $(\neg \text{IrrigationNeeded} \vee \neg \text{CropWheat} \vee \text{ApplyDripMethod})$, $(\text{ApplyDripMethod} \vee \neg \text{CropWheat} \vee \text{CropAtRisk})$, (SoilDry) , and (CropWheat) . This clause form is what the resolution procedure in part (b) operates on.

(b) Using Resolution Refutation, prove the theorem ‘ApplyDripMethod’. Negate the goal, combine with the KB in CNF, and derive the empty clause $\square$. Draw the resolution proof tree.

Resolution refutation proves a theorem by assuming its negation, adding that negated goal to the CNF knowledge base, and showing that the resulting clause set is unsatisfiable, which is demonstrated by deriving the empty clause $\square$. Here the theorem to be proved is ApplyDripMethod , so the negated goal $\neg \text{ApplyDripMethod}$ is added as an additional unit clause.

The working clause set is: $C1 = (\neg \text{SoilDry} \vee \text{IrrigationNeeded})$; $C2 = (\neg \text{IrrigationNeeded} \vee \neg \text{CropWheat} \vee \text{ApplyDripMethod})$; $C3 = (\text{SoilDry})$; $C4 = (\text{CropWheat})$; $C5 = (\neg \text{ApplyDripMethod})$, the negated goal.

Resolution step 1: Resolve $C1$ and $C3$. The complementary literals $\neg \text{SoilDry}$ and SoilDry cancel, leaving the derived unit clause $C6 = (\text{IrrigationNeeded})$.

Resolution step 2: Resolve $C6$ and $C2$. The complementary literals IrrigationNeeded and $\neg \text{IrrigationNeeded}$ cancel, leaving $C7 = (\neg \text{CropWheat} \vee \text{ApplyDripMethod})$.

Resolution step 3: Resolve $C7$ and $C4$. The complementary literals $\neg \text{CropWheat}$ and CropWheat cancel, leaving the unit clause $C8 = (\text{ApplyDripMethod})$.

Resolution step 4: Resolve $C8$ and $C5$, the negated goal. The complementary literals ApplyDripMethod and $\neg \text{ApplyDripMethod}$ cancel completely, leaving no literals at all, which is the empty clause $\square$.

The resolution proof tree therefore branches from the root empty clause upward through $C8$ and $C5$, $C8$ in turn descending from $C7$ and $C4$, $C7$ descending from $C6$ and $C2$, and $C6$ descending from $C1$ and $C3$, which are original knowledge-base clauses. Since the empty clause represents a logical

contradiction, the assumption $\neg \text{ApplyDripMethod}$ must be false, which proves by refutation that ApplyDripMethod is a logical consequence of the original knowledge base.

(c) If the fact ‘SoilDry’ is removed from the KB, analyze step-by-step how logical conclusions change. Does ‘CropAtRisk’ still follow? Apply appropriate reasoning patterns to justify.

If the fact SoilDry is removed from the knowledge base, the clause set retained for reasoning becomes: $(\neg \text{SoilDry} \vee \text{IrrigationNeeded})$, $(\neg \text{IrrigationNeeded} \vee \neg \text{CropWheat} \vee \text{ApplyDripMethod})$, $(\text{ApplyDripMethod} \vee \neg \text{CropWheat} \vee \text{CropAtRisk})$, and the single fact (CropWheat) . Without the unit clause SoilDry , the first resolution step used previously (resolving $\neg \text{SoilDry} \vee \text{IrrigationNeeded}$ against SoilDry) is no longer possible, so IrrigationNeeded can no longer be derived, and consequently ApplyDripMethod can no longer be derived through classical, sound resolution either.

Under strict classical (monotonic) first-order or propositional logic, since ApplyDripMethod is not proven true and $\neg \text{ApplyDripMethod}$ is also not proven true, the truth value of ApplyDripMethod remains genuinely unknown, and no resolution refutation can be constructed for CropAtRisk from clause $(\text{ApplyDripMethod} \vee \neg \text{CropWheat} \vee \text{CropAtRisk})$ because neither ApplyDripMethod nor $\neg \text{ApplyDripMethod}$ is available as a premise to resolve against. In this strict sense, CropAtRisk does not logically follow from the reduced knowledge base; the proof simply fails to terminate in an empty clause.

However, many practical rule-based expert systems, including typical agricultural advisory systems, use the closed-world assumption or negation-as-failure reasoning rather than strict open-world classical logic. Under this non-monotonic reasoning pattern, since ApplyDripMethod cannot be proved true from the available facts, it is treated as false by default, which makes $\neg \text{ApplyDripMethod}$ effectively true. Combined with the fact $\text{CropWheat} = \text{True}$, rule P3 ($\neg \text{ApplyDripMethod} \wedge \text{CropWheat} \rightarrow \text{CropAtRisk}$) would then fire under this default assumption, and CropAtRisk would still be derived, though through non-monotonic default reasoning rather than a sound classical proof.

In summary, whether CropAtRisk follows after removing SoilDry depends on the reasoning paradigm adopted: under pure classical monotonic logic, the conclusion is not provable and must be regarded as unknown; under the closed-world, negation-as-failure convention commonly used in practical expert systems, CropAtRisk is still inferred, but the justification is weaker and default in nature, and could be retracted if new information about ApplyDripMethod later becomes available, which is precisely the risk of non-monotonic reasoning.

Case Study 4 — Wumpus World Knowledge Agent

An AI agent navigating the Wumpus World perceives: Stench at [1,2], Breeze at [1,1], Glitter at [2,2]. The agent uses a propositional KB with these rules:

- 1) $\text{Stench}[1,2] \rightarrow \text{Wumpus is adjacent to cell}[1,2]$
- 2) $\text{Breeze}[1,1] \rightarrow \text{Pit is adjacent to cell}[1,1]$
- 3) $\text{Glitter}[x,y] \rightarrow \text{Gold is at}[x,y]$
- 4) $\neg \text{Wumpus}[x,y] \wedge \neg \text{Pit}[x,y] \rightarrow \text{Safe}[x,y]$

(a) Construct the agent's belief state from its percepts. Apply Modus Ponens systematically to derive new knowledge facts. List all derived conclusions.

The agent's percepts at the current time step are Stench at [1,2], Breeze at [1,1], and Glitter at [2,2]. These are represented as the propositional facts $\text{Stench}[1,2] = \text{True}$, $\text{Breeze}[1,1] = \text{True}$, and $\text{Glitter}[2,2] = \text{True}$, forming the initial belief state of the agent before any inference is applied.

Applying Modus Ponens, the inference rule that states from P and $P \rightarrow Q$ one may conclude Q , to each of the four given rules in turn: Rule 1 ($\text{Stench}[1,2] \rightarrow \text{WumpusAdjacent}[1,2]$) combined with the fact $\text{Stench}[1,2] = \text{True}$ yields the new derived fact $\text{WumpusAdjacent}[1,2] = \text{True}$, meaning the Wumpus occupies one of the cells adjacent to [1,2].

Rule 2 ($\text{Breeze}[1,1] \rightarrow \text{PitAdjacent}[1,1]$) combined with the fact $\text{Breeze}[1,1] = \text{True}$ yields the new derived fact $\text{PitAdjacent}[1,1] = \text{True}$, meaning a pit occupies one of the cells adjacent to [1,1].

Rule 3 ($\text{Glitter}[x,y] \rightarrow \text{GoldAt}[x,y]$), instantiated with $x=2$, $y=2$, combined with the fact $\text{Glitter}[2,2] = \text{True}$, yields the new derived fact $\text{GoldAt}[2,2] = \text{True}$.

Rule 4 ($\neg \text{Wumpus}[x,y] \wedge \neg \text{Pit}[x,y] \rightarrow \text{Safe}[x,y]$) cannot yet fire for any specific cell because the agent has not yet established that both the Wumpus and the Pit are absent from any particular unvisited cell; it will become usable only once enough adjacency information is combined in part (b).

The complete list of facts newly derived in the agent's belief state after this round of Modus Ponens application is therefore: $\text{WumpusAdjacent}[1,2] = \text{True}$, $\text{PitAdjacent}[1,1] = \text{True}$, and $\text{GoldAt}[2,2] = \text{True}$, in addition to the three original percept facts. This belief state forms the foundation for the forward-chaining location analysis carried out in part (b).

(b) Using Forward Chaining from the agent's percepts, identify the possible locations of the Wumpus and Pit. Show each reasoning step precisely.

Forward chaining is now applied iteratively, using the derived adjacency facts from part (a) together with basic knowledge of grid adjacency, to progressively narrow down the possible locations of the Wumpus and the Pit.

Step 1: Since $\text{WumpusAdjacent}[1,2] = \text{True}$, the Wumpus must occupy one of the cells orthogonally adjacent to $[1,2]$, namely $[1,1]$, $[1,3]$, or $[2,2]$ (the fourth neighbour, $[0,2]$, does not exist on the grid and is discarded).

Step 2: The agent has already visited $[1,1]$ as its starting square and survived without any adverse event, which is standard background knowledge in the Wumpus World that the starting square is always guaranteed safe. This lets the agent apply Rule 4 to conclude $\text{Safe}[1,1] = \text{True}$ and, further, that the Wumpus is not at $[1,1]$. The candidate set for the Wumpus therefore reduces to $\{[1,3], [2,2]\}$.

Step 3: Since $\text{PitAdjacent}[1,1] = \text{True}$, the pit must occupy one of the cells orthogonally adjacent to $[1,1]$, namely $[1,2]$ or $[2,1]$ (the other two neighbours of $[1,1]$ are outside the grid and discarded). Because the agent has already moved to $[1,2]$ and perceived a stench there without falling into a pit, $[1,2]$ is confirmed not to contain a pit, so $\text{Safe}[1,2] = \text{True}$ with respect to the pit hazard. The candidate set for the pit therefore reduces to the single cell $\{[2,1]\}$, and forward chaining concludes $\text{Pit}[2,1] = \text{True}$.

Step 4: With the pit now located at $[2,1]$, and using the fact that $[1,2]$ has been safely visited (ruling out both Wumpus and Pit there), the Wumpus location remains ambiguous between $[1,3]$ and $[2,2]$ until the agent gathers further percepts, for example by moving to an additional adjacent cell such as $[2,1]$ or $[1,3]$ and checking for the presence or absence of stench, which would allow another application of forward chaining to eliminate one of the two remaining candidates.

In summary, forward chaining from the percepts precisely locates the Pit at $[2,1]$, while it narrows the Wumpus to one of two candidate cells, $\{[1,3], [2,2]\}$, reflecting the general property of the Wumpus World that logical inference from a single stench percept alone is often sufficient to bound but not always uniquely pinpoint the Wumpus's exact location.

(c) Evaluate whether the path $[1,1] \rightarrow [1,2] \rightarrow [2,2]$ is safe for the agent to collect the gold.

Use the knowledge derived in (a) and (b) to justify your conclusion with logical inference.

To evaluate whether the path $[1,1] \rightarrow [1,2] \rightarrow [2,2]$ is safe, each cell on the path must be checked against Rule 4, $\text{Safe}[x,y] \leftarrow \neg \text{Wumpus}[x,y] \wedge \neg \text{Pit}[x,y]$, using the conclusions derived in parts (a) and (b).

Cell $[1,1]$: this is the agent's starting square. By the standard convention of the Wumpus World that the starting cell is always free of hazards, and confirmed by the fact that the agent occupies it without harm, both $\neg \text{Wumpus}[1,1]$ and $\neg \text{Pit}[1,1]$ hold, so $\text{Safe}[1,1] = \text{True}$.

Cell $[1,2]$: the agent has already moved to this cell and perceived $\text{Stench}[1,2]$ and, implicitly, no Breeze there (since $\text{Breeze}[1,1]$ was reported at the adjacent cell $[1,1]$ rather than at $[1,2]$ itself, and no breeze percept is given for $[1,2]$). The absence of a breeze percept at $[1,2]$ combined with the agent surviving its visit there confirms $\neg \text{Pit}[1,2]$. Although a stench is perceived, meaning the Wumpus is adjacent to $[1,2]$, the agent itself is not harmed while standing on $[1,2]$, confirming $\neg \text{Wumpus}[1,2]$ as well (a stench indicates Wumpus in a neighbouring cell, not on the current cell). Therefore $\text{Safe}[1,2] = \text{True}$, consistent with part (b).

Cell $[2,2]$: from part (b), the Wumpus candidate set was narrowed to $\{[1,3], [2,2]\}$ but not resolved to a single cell; $[2,2]$ therefore remains a genuine possibility for the Wumpus's location and has not been excluded by any derived fact. Because Rule 4 requires $\neg \text{Wumpus}[2,2]$ to be established before $\text{Safe}[2,2]$ can be concluded, and this negation has not been proved, $\text{Safe}[2,2]$ cannot be logically asserted from the current knowledge base.

Conclusion: the first two segments of the path, $[1,1] \rightarrow [1,2]$, are logically confirmed safe and may be traversed with confidence. However, the final segment into $[2,2]$ cannot be certified safe using the knowledge derived so far, since $[2,2]$ remains an unresolved candidate for the Wumpus's location. A cautious, logically sound agent should therefore not proceed directly to $[2,2]$ to collect the glittering gold; instead, it should first gather an additional percept, for example by visiting $[1,3]$ (or another safe adjacent cell) to check for the presence or absence of stench there, which through Rule 1 and forward chaining would allow the Wumpus's location to be pinned down definitively, after which $\text{Safe}[2,2]$ could either be confirmed or the path safely avoided.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding ; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand